Report: PDDL Quest Series Generator

Autorzy:

- Paulina Hładki (nr indeksu: _____)
- Marta Jędrzejczak (nr indeksu: _____)
- Szymon Szymankiewicz (151821)
- Dominik Maćkowiak (151915)

1. Artifacts

The project contains the four required elements:

- Story generator code: `questgen/generator.py`, supported by `questgen/domain.py` and `questgen/pddl.py`.
- Quest definition folder: `quests/generated/ash_bell/`.
- Story playback code: `questgen/play.py`.
- Project report: this file.

The generated folder contains:

- `domain.pddl` - the shared world domain.
- `prompt.txt` - the original English prompt.
- `series.json` - the full generated series.
- `quest_01_quest-1.json`, `quest_02_quest-2.json`, `quest_03_quest-3.json` - quest descriptions, objects, NPCs, and dialogue.
- `quest_01_quest-1.pddl`, `quest_02_quest-2.pddl`, `quest_03_quest-3.pddl` - planning problems.
- `quest_*.plan` - plans verified by the local STRIPS solver.
- `generation_meta.json` - generation metadata.

2. Generation Method

The method is hybrid. A local language model creates a compact dramatic seed, while deterministic symbolic code compiles that seed into JSON and PDDL. This avoids asking the model to write raw PDDL, which is fragile. The language model contributes title, premise, motifs, quest titles, summaries, and a twist. The generator then builds a validated quest structure with locations, items, NPCs, enemies, and steps.

The first attempt used `qwen3.6:latest`, but that model was too heavy for the laptop in this setup. It repeatedly timed out and also produced long thinking output before returning useful JSON. The final run uses the much smaller `gemma3:4b` model. It completed successfully:

```
{
  "model": "gemma3:4b",
  "used_fallback": false,
  "generation_strategy": "ollama_compact_draft_then_symbolic_pddl_compilation"
}
```

The generator still supports three modes:

- `compact` - default mode; asks the LLM for a short dramatic seed, then compiles it symbolically.
- `full` - experimental mode; asks the LLM for a full quest schema.

- `fallback` - deterministic English fallback used when the model is unavailable.

3. Shared PDDL Domain

The shared domain uses these types:

- `location`
- `item`
- `npc`
- `enemy`
- `gate`
- `stage`

The STRIPS actions represent the actions available in the text game:

- `travel`
- `take`
- `talk`
- `give`
- `unlock`
- `fight`
- `ritual`
- `examine-location`
- `examine-item-at`
- `examine-carried-item`
- `examine-npc`
- `examine-enemy`
- `reveal-alternate-path`
- `travel-alternate`
- `take-optional`
- `press-npc`
- `brawl`
- `use-healing-item`

Each action has preconditions and effects. The intended narrative order is represented with stage predicates such as `current-stage`, `next-stage`, `travel-step`, `take-step`, `fight-step`, and `ritual-step`. This keeps the quest as a planning problem while preventing the solver from skipping the intended story beat order. The `examine-*` actions add optional inspection choices. The consequence actions add decisions with state changes: hidden routes can be revealed, optional items can be taken, NPCs can be provoked, optional enemies can become hostile, brawls can wound the player, and healing items can remove the wound.

4. Repair and Verification

The generator does not trust model output directly. It normalizes IDs, filters unsupported actions, adds missing objects, and compiles quest steps into initial PDDL facts.

The repair pass can:

- add missing locations, NPCs, items, enemies, and gates,
- infer `start_location`,

- add `path` facts for movement,
- add `locked-gate` facts for locked passages,
- add `hidden-route`, `optional-item`, `healing-item`, `provocation-open`, and `optional-enemy` facts for decision consequences,
- infer carried items with `has(item)` when an item continues from a previous quest,
- add fallback dialogue for NPCs,
- compare the solver plan against the expected narrative plan.

Final verification:

```
OK   quest_01_quest-1.pddl: 5 actions
OK   quest_02_quest-2.pddl: 7 actions
OK   quest_03_quest-3.pddl: 6 actions
```

5. Generated Series

Original prompt:

```
After years away, the hero returns to their childhood village and discovers
that an ancient bell beneath the mine is waking ash-born shadows.
The story should feel like a dark fairy tale, but end with hope for the village.
```

Generated series title:

```
Echoes of the Ashwood
```

LLM-generated dramatic seed:

- Motifs: `loss`, `memory`, `corruption`
- Twist: the shadows are not malevolent, but echoes of a forgotten pact made to protect the village from a far greater threat.

Quest sequence:

1. `The Silent Toll` - the call to adventure; the hero speaks with Elder Mira, finds the rusted medallion, and takes the silver knife.
2. `Beneath Blackstone` - the threshold; the hero gives the medallion to Iren the Cartographer, obtains the vein map and key, and opens the mine gate.
3. `The Bloom of Remembrance` - the ordeal and return; the hero defeats the ash guardian, listens to the bellkeeper's shade, retrieves the bell heart, and performs the rite of silence.

6. Multi-Prompt Experiment

To test the robustness of the generator, three different English prompts were fed to the `gemma3:4b` model. Two runs used the default `compact` mode, and one used the experimental `full` mode. All outputs were verified by the local STRIPS solver and then played through automatically with the text player.

Prompt theme	Mode	Generated series title	LLM succeeded?	Underlying quest structure
Disgraced naval captain + cursed sextant	`compact`	*The Salt-Kissed Curse*	Yes (`used_fallback: false`)	Identical to fallback template
Astronaut on a Jovian moon + alien signal	`full`	*Echo of the Ashen Bell*	No (`used_fallback: true`)	Fallback template (LLM produced an unsolvable schema)
Musician inherits a haunted opera house	`compact`	*Echoes of Porcelain*	Yes (`used_fallback: false`)	Identical to fallback template

Observation 1 — Narrative skin changes, structure does not. In every successful `compact` run, `gemma3:4b` produced a new dramatic seed (title, premise, motifs, twist, and quest titles), but the symbolic compiler then built the actual quest steps from the same deterministic fallback template. Consequently, the locations, items, NPCs, enemies, and step order remained identical across all three prompts. The model decorated the skeleton with different words, but it did not design a new skeleton.

Observation 2 — `full` mode is impractical for this model. When asked for a complete quest schema in `full` mode, `gemma3:4b` emitted a structure that failed the planner's solvability check even after the repair pass. The generator automatically rejected the draft and fell back to the deterministic template. This confirms that asking a 4-billion-parameter model to write valid, solvable PDDL-compatible quest graphs is currently unreliable.

Observation 3 — All generated series are playable. Regardless of whether the LLM contributed or not, every quest in every series passed the STRIPS solver, and the `--auto` playthrough completed each series without errors.

7. Playback

The text player loads both JSON and PDDL. The JSON provides descriptions, names, narration, and dialogue. The PDDL state determines which actions are currently available. The menu is generated from applicable grounded STRIPS actions, not from a hand-written script.

The first implementation was too linear: each state usually exposed only one action. The current version fixes that with optional STRIPS actions and consequence actions. For example, the first state now offers:

```
Available actions:
  1. Talk to: Elder Mira
  2. Examine: Heatherfall
  3. Observe: Elder Mira
```

Later choices can change the game state. Examples:

```
Available actions:
  1. Go to: Ancestor Shrine
```

2. Examine: Old Well
3. Examine carried item: Rusted Medallion
4. Reveal hidden path with: Rusted Medallion

In the second quest, pressing Iren creates a hostile guard. Fighting the guard wounds the player, and using `Bruisewort Tonic` removes the wound:

```
Pressed too hard, Iren snaps her map case shut and signals the oathbound guard.
Threats: Oathbound Guard (hostile)

You beat back Oathbound Guard, but the struggle leaves you wounded.
Status: wounded

You use Bruisewort Tonic. The wound stops bleeding.
```

Example command:

```
python3 -B -m questgen.play --series quests/generated/ash_bell
```

Automatic full playthrough:

```
python3 -B -m questgen.play --series quests/generated/ash_bell --auto
```

8. Strengths and Weaknesses

Strengths:

- The generated PDDL is controlled and easy to repair.
- Every quest is verified by a local STRIPS planner.
- JSON and PDDL are compiled from the same quest steps.
- The player uses the same action semantics as the planner.
- The player has optional exploration choices and consequential side choices instead of a single forced action at most states.
- `gemma3:4b` is small enough to run locally and was fast enough for this task.

Weaknesses:

- The symbolic compiler limits narrative freedom compared with full free-form LLM generation.
- Stage predicates still guide the critical quest path through a canonical narrative order.
- Cross-quest state transfer is simplified; important carried items are inferred per quest.
- Optional consequences affect local state but do not currently branch the final quest objective.
- The `full` LLM mode remains experimental and may produce invalid schemas.
- `gemma3:4b` in `compact` mode only changes the narrative "skin"; it does not generate structurally different quests. The underlying locations, items, NPCs, and step order always fall back to the deterministic template.

9. Notes on Model Choice

Using English prompts improved reliability and made `gemma3:4b` sufficient. The larger `qwen3.6:latest` model was technically installed, but it was not practical here because it timed out even on compact prompts. The final generated series is therefore a real Ollama-assisted run using the smaller downloaded model, not a fallback-only result.

10. Podział prac

Osoba	Główny obszar	Szczegółowy opis wykonanych zadań
Paulina Hładki	**Domena PDDL i solver STRIPS**	Zaprojektowanie wspólnej domeny `mythic_quest` (typy, predykaty, akcje STRIPS) w pliku `questgen/domain.py`. Implementacja parsera PDDL i forward plannera (BFS) w pliku `questgen/pddl.py` — parsowanie, grounding akcji, solver, weryfikacja planów. Stworzenie testów jednostkowych dla parsera i solvera.
Marta Jędrzejczak	**Generator fabuł i integracja LLM**	Implementacja generatora w pliku `questgen/generator.py` — integracja z API Ollamy, kompilacja *dramatic seed* do PDDL/JSON, mechanizm naprawy `repair_quest_schema`, weryfikacja `compile_and_verify`. Wygenerowanie pierwszej działającej serii `ash_bell` w trybie `compact` z modelem `gemma3:4b`.
Szymon Szymankiewicz	**Interfejs tekstowy (player)**	Implementacja silnika gry w pliku `questgen/play.py` — ładowanie JSON/PDDL, generowanie menu z grounded STRIPS actions, system narracji, dialogów, konsekwencji (rany, wrogowie, opcjonalne przedmioty). Dodanie opcji `--auto` do automatycznego przechodzenia questów. Testy dla helperów playera.

Osoba	Główny obszar	Szczegółowy opis wykonanych zadań
Dominik Maćkowiak	**Testy, weryfikacja i raport**	Stworzenie testów jednostkowych w katalogu `tests/` (w tym `test_output_files.py` i `test_integration.py`). Weryfikacja wszystkich wygenerowanych serii questów. Napisanie i skład raportu (`raport.md` → PDF).